

Атрофовизуализация — Dataset Manager

Настольное приложение для работы с датасетами ОКТ-изображений (оптическая когерентная томография кожи). Позволяет загружать снимки, организовывать их в датасеты, выполнять ручную разметку сеткой и вычислять локальную светимость по выделенным областям.

Как устроен проект

Приложение состоит из трёх слоёв, запускаемых совместно.

Tauri (Rust) — оболочка десктопного приложения. Встраивает WebView и оборачивает его в нативный процесс ОС. Никакого дополнительного рантайма на машине пользователя не требует. Исполняемый файл при запуске запускает backend и открывает окно с интерфейсом.

Frontend (Vue 3 + TypeScript + Vite) — одностраничное приложение, которое работает внутри WebView. Общается с бэкендом через обычный `fetch()` на `localhost:8000`. Состояние UI хранится в Pinia-сторах. Основные экраны:

- `MainMenu.vue` — список датасетов
- `ImageView.vue` — список изображений внутри датасета
- `AnalysisView.vue` — рабочая область анализа
- `ManualAnalysis.vue` — разметка сеткой, таблица светимости, статистика по категориям
- `InteractiveImage.vue` — канвас с перетаскиваемыми линиями сетки

Backend (FastAPI + Python 3.12) — HTTP-сервер на порту 8000. Хранит метаданные в SQLite через SQLAlchemy (async). Файлы изображений лежат на диске в `uploads/images/{dataset_id}/`. Обработка изображений происходит в памяти через OpenCV и NumPy — промежуточные результаты на диск не пишутся. Структура:

```
app/
├── api/v1/endpoints/
│   ├── IO/           # датасеты, изображения, архивы
│   └── analysis/     # gaussian_blur, calculate_mean_lines
├── service/
│   ├── IO/           # dataset_service, image_service, file_services, archive_service
│   └── computation/  # filter_service, brightness_service
├── models/           # SQLAlchemy ORM: Dataset, Image, Results
└── schemas/          # Pydantic-схемы запросов/ответов
```

Результаты ручного анализа (координаты сетки, значения светимости, категории ячеек) сохраняются в таблицу `Results` как JSON.

Архивирование (экспорт/импорт датасета в ZIP) реализовано через `TaskManager` — задача уходит в фоновый `ThreadPoolExecutor`, статус можно опросить через SSE-endpoint.

Скриншоты

Список датасетов

Атрофовизуализация

Мои документы

Настройки

Мои документы

Импортировать документ

+ Создать документ

100% замаскировано

Склеродермия новая

Склеродермия от Анастасии

22 апр. 2020 г.

100% замаскировано

атрофия

атрофия от Екатерины Владимовны

22 апр. 2020 г.

100% замаскировано

Здоровая тест

тест

16 апр. 2020 г.

100% замаскировано

Здоровая кожа (Imported 28.02 00:21)
(Imported 14.04 22:34)

жестерная оценка

100% замаскировано

Склеродермия (Imported 28.02 00:22)
(Imported 14.04 21:40)

жестерная оценка

Список изображений в датасете

Атрофовизуализация

Мои документы

Настройки

← Назад к документам

Изображения документа #2

Загрузить изображения

Обновить

Выбрать все

Удалить выбранные

Анализ

Slide51.png

ID: 200

Slide50.png

ID: 199

Slide49.png

ID: 198

Slide48.png

ID: 197

Slide47.png

ID: 196

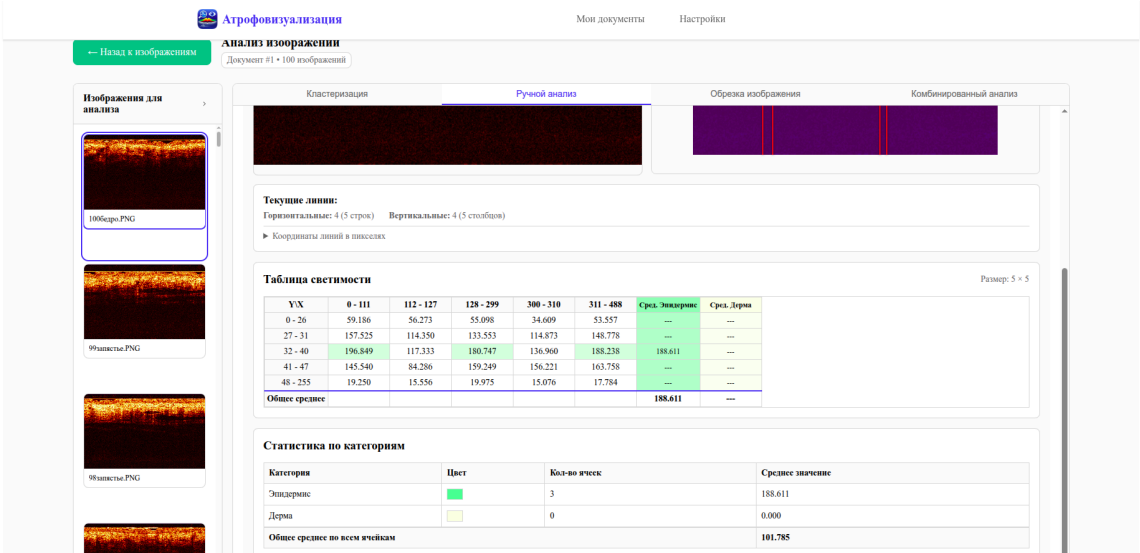
Slide46.png

ID: 195

Разметка сеткой и применение фильтра Гаусса



Таблица светимости и статистика по категориям



Требования

Компонент	Версия
Python	3.10 – 3.12
Node.js	20+
Rust (stable)	последняя стабильная
uv	любая

Linux — дополнительные системные пакеты:

```
sudo apt-get install -y \
  libwebkit2gtk-4.1-dev \
  build-essential curl wget file \
  libssl-dev \
  libayatana-appindicator3-dev \
  librsvg2-dev
```

Установка из готового релиза

Windows

В папке с дистрибутивом находится файл `Atrofovizualizatia.exe` — установщик на базе Inno Setup.

1. Запустите `Atrofovizualizatia.exe`, следуйте инструкциям установщика.

Если используется portable-версия (папка `release/windows/`):

```
release/windows/
├─ dataset-manager.exe ← исполняемый файл
└─ backend/           ← Python-бэкенд
```

Перед первым запуском установите зависимости бэкенда:

```
cd backend
# Установить uv, если нет:
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
uv sync
```

Затем запустите `dataset-manager.exe`.

Linux

В папке с дистрибутивом находится папка `release/linux/`.

1. Откройте папку `release/linux/` в терминале и выполните:

```
./install.sh # установит uv и зависимости backend в .venv
./start.sh   # запустит backend + GUI
```

`start.sh` автоматически запускает backend в фоне перед открытием окна и останавливает его при закрытии приложения.

Запустить только backend (без GUI, для отладки):

```
./start-backend-only.sh
# API доступен на http://127.0.0.1:8000
# Swagger UI: http://127.0.0.1:8000/docs
```

Ручная сборка из исходников

1. Клонировать репозиторий с подмодулями

```
git clone --recurse-submodules <url>
cd tauri-dataset-manager-app
```

2. Установить зависимости backend

```
cd backend
uv sync
cd ..
```

Если `uv` не установлен:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

3. Linux — сборка через скрипт

```
chmod +x scripts/build-auri.sh scripts/create-release-scripts.sh
cd scripts
./build-auri.sh          # сборка release-бинарника
./create-release-scripts.sh # генерирует install.sh / start.sh в release/linux/
```

Результат окажется в:

```
release/linux/
├─ dataset-manager      ← бинарник Tauri
├─ backend/             ← копия Python-бэкенда
├─ install.sh
├─ start.sh
└─ start-backend-only.sh
```

4. Windows — сборка через PowerShell

```
cd scripts
.\build-auri.ps1
.\create-release-scripts.ps1
```

Для сборки установщика нужен [Inno Setup](#):

```
iscc scripts/installer.iss
```

Установщик появится в `release/Atrofovizualizatia.exe` .

5. Ручная сборка frontend (без скриптов)

```
cd frontend
npm install
npm run build          # или: npm run tauri:build
```

Бандл Tauri кладёт артефакты в:

- Linux: frontend/src-tauri/target/release/bundle/
- Windows: frontend/src-tauri/target/release/bundle/nsis/ и bundle/msi/

Режим разработки

Запустить backend и frontend отдельно, с hot-reload:

Backend:

```
cd backend
uv run python main.py
# http://127.0.0.1:8000/docs
```

Frontend (браузер):

```
cd frontend
npm install
npm run dev          # Vite dev server, http://localhost:5173
```

Frontend (Tauri dev-режим):

```
cd frontend
npm run tauri:dev
```

Тесты (backend)

```
cd backend
.venv/bin/pytest -v
```

С отчётом покрытия:

```
.venv/bin/pytest --cov=app --cov-report=term-missing
```

Тесты используют SQLite in-memory, реальный диск не трогают (кроме

test_archive_export_import_flow , который создаёт временные файлы в uploads/ и убирает их в finally).

Текущее покрытие: **70%**. Хорошо покрыты сервисы вычислений (`brightness_service` 97%, `filter_service` 95%), IO-сервисы и эндпоинты ручного анализа.

CI / GitHub Actions

Файл: `.github/workflows/build-release-simple.yml`

Сборка запускается при push в ветки `manual-only` или `tauri` , а также вручную (`workflow_dispatch`).

Job	Платформа	Артефакты
build-linux	ubuntu-latest	release-linux
build-windows	windows-latest	release-windows-portable, release-windows-installer

Для Linux: устанавливает системные зависимости `webkit2gtk`, запускает `build-tauri.sh` + `create-release-scripts.sh` .

Для Windows: запускает `build-tauri.ps1` , затем `create-release-scripts.ps1` , затем собирает установщик через Inno Setup (`iscc scripts/installer.iss`).

Руководство пользователя

1. Управление датасетами

Главное окно (кнопка **«Мои датасеты»** в шапке) отображает карточки документов с названием, описанием, датой создания и количеством изображений.

- **«+ Создать документ»** — открывает форму, где нужно ввести название и описание нового датасета.
- **«Импортировать документ»** — принимает ZIP-архив, ранее экспортированный из приложения; создаёт новый датасет со всеми изображениями и результатами анализа.

2. Работа со списком изображений

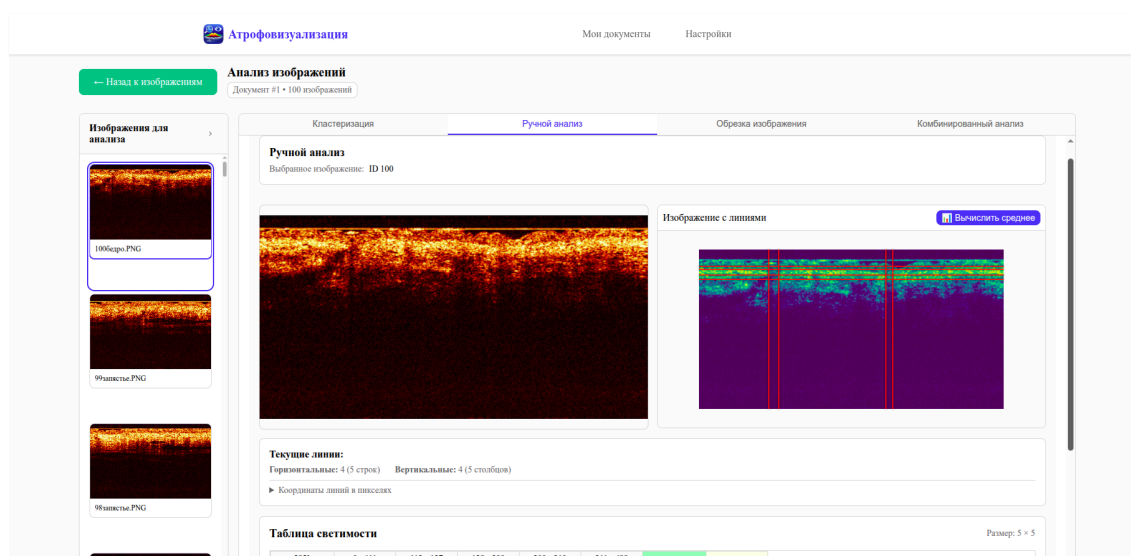
После нажатия на карточку датасета открывается список загруженных снимков.

- **«Загрузить изображения»** — открывает диалог выбора файлов (JPEG, PNG, WebP, GIF, до 10 МБ). Можно выбрать несколько файлов сразу.
- **«Обновить»** — обновляет список без перезагрузки страницы.
- Чекбоксы на карточках или кнопка **«Выбрать все»** — выделяют снимки для дальнейших действий.
- **«Удалить выбранные»** — удаляет отмеченные изображения вместе с файлами на диске.
- **«Анализ»** — переходит в рабочую область для выделенных снимков.

3. Рабочая область

Слева — панель со списком выбранных изображений для быстрого переключения между ними. Щелчок по миниатюре загружает нужный снимок в область анализа.

4. Ручной анализ



Инструмент предназначен для нанесения сетки на изображение и расчёта средней светимости в каждой ячейке.

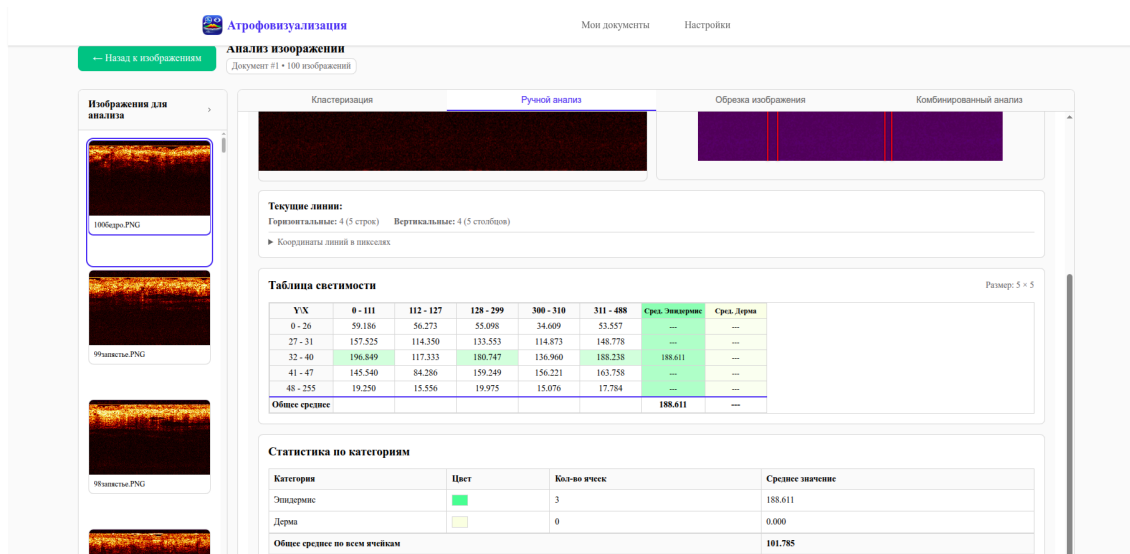
Нанесение линий:

1. Нажмите **правой кнопкой мыши** на свободной области изображения — откроется контекстное меню.
2. Выберите **«Добавить горизонтальную линию»** или **«Добавить вертикальную линию»**.
3. Перетащите линию мышью, совместив её с границей анатомической структуры на снимке.
4. Чтобы удалить ошибочно добавленную линию — нажмите на неё **правой кнопкой мыши** и выберите **«Удалить»**.

Текущее количество линий и координаты в пикселях отображаются в блоке **«Текущие линии»** под изображением.

Вычисление светимости:

Нажмите **«Вычислить среднее»**. Сервер применяет сглаживающий фильтр Гаусса и рассчитывает среднюю яркость каждой ячейки по L-каналу цветового пространства Lab. Результаты появляются в **Таблице светимости**.



Назначение категорий:

1. Нажмите **«Управление категориями»** и создайте нужные категории, задав им название и цвет (например, «Эпидермис» — зелёный, «Дерма» — жёлтый).
2. Щёлкните по ячейке в **Таблице светимости** — появится выпадающий список для выбора категории. Ячейка окрасится в цвет категории.
3. Блок **«Статистика по категориям»** автоматически показывает количество ячеек и среднюю светимость для каждой назначенной категории.

Сохранение:

- Кнопка **«Сохранить»** — фиксирует текущую разметку (положение линий и назначенные категории) в базе данных.
- При попытке переключиться на другое изображение без сохранения появится предупреждение с кнопками **«Сохранить»** и **«Отменить»**.

Экспорт датасета:

На странице со списком изображений нажмите **«Экспорт»** — приложение сформирует ZIP-архив со всеми снимками и результатами анализа. Архив можно импортировать на другой машине через кнопку **«Импортировать документ»** на главном экране.

5. Настройки

Вкладка **«Настройки»** в шапке позволяет переключить тему оформления (светлая / тёмная) и изменить отображаемое имя пользователя.

Структура репозитория

```
tauri-dataset-manager-app/  
├─ backend/                # FastAPI-приложение  
│   └─ app/  
│       └─ api/v1/endpoints/ # IO и analysis эндпоинты  
│       └─ service/         # бизнес-логика
```

```
| | | └─ models/           # SQLAlchemy ORM
| | |   └─ schemas/        # Pydantic
| | └─ tests/              # pytest
| | └─ uploads/            # хранилище файлов (создаётся при запуске)
| | └─ main.py
| | └─ pyproject.toml
└─ frontend/               # Vue 3 + Tauri
    └─ src/
        └─ api/            # fetch-обёртки (datasets, images, manual)
        └─ components/     # Vue-компоненты
        └─ views/          # страницы (маршруты)
        └─ stores/         # Pinia
    └─ src-tauri/          # Rust/Tauri конфигурация и код
    └─ package.json
└─ scripts/               # build-tauri.sh/.ps1, installer.iss, ...
└─ docs/                  # скриншоты для README
└─ .github/workflows/     # CI
```

API

После запуска backend документация доступна на:

- Swagger UI: <http://127.0.0.1:8000/docs>
- ReDoc: <http://127.0.0.1:8000/redoc>

Все маршруты начинаются с `/api/v1` . Основные группы:

Префикс	Описание
<code>/api/v1/I0/dataset</code>	CRUD датасетов
<code>/api/v1/I0/image</code>	загрузка, скачивание, удаление, инфо по изображению
<code>/api/v1/I0/archive</code>	экспорт/импорт датасета ZIP
<code>/api/v1/analysis/manual</code>	фильтр Гаусса, расчёт светимости по сетке
<code>/api/v1/tasks</code>	статус фоновых задач